

A 送分题

$O(n^2)$ 或者 $O(n)$ 的模拟都可以，也可以把所有数异或起来。因为偶数个相同的数 x 按位异或得到的值为0，奇数个相同的数异或起来仍为 x 。只有一种数出现了奇数次。

```
#include <iostream>

int main() {
    int n;
    std::cin >> n;

    int res = 0; // 初始化为 0 (因为 0 ^ x = x)
    for (int i = 0; i < n; ++i) {
        int x;
        std::cin >> x;
        res ^= x; // 异或运算
    }

    std::cout << res << std::endl;
    return 0;
}
```

B、LCM：盲猜！

为方便描述，设第 i ($i > 0$)秒开始时，我们进行按按钮操作，第 i ($i \geq 0$)秒结束时，我们思考完成。Alice的数为 n ，Bob的数为 m

$A[x][i]$ 是一个集合(set)，表示若 $n = x$ ，第 i 秒结束时Alice认为 m 的可能值的集合。

$B[x][i]$ 是一个集合(set)，表示若 $m = x$ ，第 i 秒结束时Bob认为 n 的可能值的集合。

对于所有 $lcm(n, m)$ 的约数，我们可以预处理出 $A[x][0]$ 和 $B[x][0]$

若 $\min\{A[x][i]\} \geq n$ 或 $\max\{A[x][i]\} \leq n$ ，则 A 在 $i + 1$ 秒开始时会按下按钮。 $b[x][i]$ 亦然。

我们记满足这样性质的 AB ， x ， i 为 $f(A, x, i) = 1$ 或 $f(B, x, i) = 1$

$A[x][i + 1] = A[x][i] - \{y | y \in A[x][i] \&\& f(B, y, i) == 1\}$ ， $B[x][i + 1]$ 亦然。

当 $f(A, n, i) == 1$ or $f(B, m, i) == 1$ 第一次成立时， $i + 1$ 即为所求。

实现时可以直接模拟这个过程，复杂度为 $O(ans * d(lcm)^2)$ ， d 为约数个数。

也可以把 lcm 的约数连成一张有向图，进行拓扑排序，边数是 $O(d(lcm)^2)$ 的。

```

#include<bits/stdc++.h>
#define fi first
#define se second
#define int long long
using namespace std;
typedef pair<int,int> pii;
const int maxn=2000+10;
int inf=1e9;
int n,m,k;
int les[maxn],gr[maxn];
vector<int> ver[maxn];
signed main(){
    ios::sync_with_stdio(0);
    cin>>n>>m;
    k=n/___gcd(n,m)*m;
    vector<int> fact;
    for(int i=1;i*i<=k;i++){
        if(k%i==0){
            fact.push_back(i);
            if(i*i!=k){
                fact.push_back(k/i);
            }
        }
    }
    sort(fact.begin(),fact.end());
    for(int i=0;i<fact.size();i++){
        for(int j=0;j<fact.size();j++){
            if(fact[i]/___gcd(fact[i],fact[j])*fact[j]!=k) continue;
            if(j>i) gr[i]++;
            if(j<i) les[i]++;
            ver[j].push_back(i);
        }
    }
    vector<int> last;
    for(int i=0;i<fact.size();i++){
        if(gr[i]==0||les[i]==0){
            last.push_back(i);
        }
    }
    for(int i=1;i<=fact.size();i++){
        vector<int> n1;
        for(auto u:last){
            if(fact[u]==n||fact[u]==m){
                cout<<i<<"\n";
                return 0;
            }
            for(auto v:ver[u]){
                if(u>v){
                    gr[v]--;
                    if(gr[v]==0&&les[v]>0){
                        n1.push_back(v);
                    }
                }
                if(u<v){
                    les[v]--;
                    if(les[v]==0&&gr[v]>0){
                        n1.push_back(v);
                    }
                }
            }
        }
        last=n1;
    }
}

```

```

    cout<<"-1\n";
    return 0;
}

```

C-小树的世界

```

print(int(input()) * 2 + 2)

```

D-最小值

两个数组均正序排列可以得到 $\sum_{i=1}^n \max(a_i, b_i)$ 的最小值

一个正序，一个逆序可以得到 $\sum_{i=1}^n \min(a_i, b_i)$ 的最小值

```

void Solve()
{
    int n;
    cin >> n;
    vector<int> a(n + 1), b(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    for (int i = 1; i <= n; i++)
        cin >> b[i];
    sort(a.begin() + 1, a.end());
    sort(b.begin() + 1, b.end(), greater<int>());
    ll ans1 = 0;
    for (int i = 1; i <= n; i++)
        ans1 += min(a[i], b[i]);
    sort(b.begin() + 1, b.end());
    ll ans2 = 0;
    for (int i = 1; i <= n; i++)
        ans2 += max(a[i], b[i]);
    cout << ans1 << " " << ans2 << "\n";
}

```

E-洗牌大师

Solution

经过仔细分析，可以证明最终洗完后“顶牌”来源于原序列中某个位置，且该位置可以用下面的互递归关系求出。我们把整个洗牌过程看作“不断压缩”牌堆：

1. 对于奇数轮（记作 $f_{\text{odd}}(n)$ ），将第 1、3、5...位置的牌堆叠后，牌堆数变为 $\text{ceil}(n/2)$ ，新牌堆的顶牌等于这一对中原本处于左侧的牌。因此可得

$$f_{\text{odd}}(n) = 2 \cdot f_{\text{even}}(\text{ceil}(n/2)) - 1.$$

2. 对于偶数轮（记作 $f_{\text{even}}(n)$ ），将第 2、4、6...位置的牌堆叠到左侧，其结果为：

如果 n 为偶，则 $f_{\text{even}}(n) = 2 \cdot f_{\text{odd}}(n/2)$;

如果 n 为奇 ($n \geq 3$)，令 $m = (n+1)/2$ ，则 $f_{\text{even}}(n) =$

$$\begin{array}{ll} 2 \cdot f_{\text{odd}}(m) & \text{if } f_{\text{odd}}(m) < m, \\ 2 \cdot m - 1 & \text{otherwise.} \end{array}$$

显然 $f_{\text{odd}}(1)=f_{\text{even}}(1)=1$ 。

为便于自底向上求解，我们建立一个“链”：令

$$A[0] = n, \quad A[i+1] = \text{ceil}(A[i] / 2)$$

直到 $A[k] = 1$ 为止。并设 $X[i] = f_{\text{odd}}(A[i])$ 、 $Y[i] = f_{\text{even}}(A[i])$ 。则有递推：

$$X[i] = 2 \cdot Y[i+1] - 1,$$

$$\text{若 } A[i] \text{ 为偶 (则 } A[i+1]=A[i]/2) : \quad Y[i] = 2 \cdot X[i+1];$$

$$\text{若 } A[i] \text{ 为奇 (则 } A[i+1]=(A[i]+1)/2) : \quad Y[i] = (2 \cdot X[i+1] \quad \text{if } X[i+1] < A[i+1], \text{ else } 2 \cdot A[i+1] - 1).$$

最终答案为 $f(n) = f_{\text{odd}}(n) = X[0]$.

考虑到 n 可达 10^{18} ，而每轮递推均将牌堆数减半，自底向上计算只需 $O(\log n)$ 次迭代，每组测试时间足够，总体复杂度 $O(T \cdot \log n)$ (约 6×10^6 次迭代)。

Code

```
#include <bits/stdc++.h>
using namespace std;
typedef unsigned long long ull;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;
    while(T--){
        ull n;
        cin >> n;
        // 建立链 A: A[0] = n, A[i+1] = ceil(A[i]/2)
        vector<ull> A;
        A.push_back(n);
        while(A.back() > 1) {
            ull cur = A.back();
            if(cur % 2 == 0)
                A.push_back(cur / 2);
            else
                A.push_back((cur + 1) / 2);
        }
        int L = A.size();
        // 定义 X[i] = f_odd(A[i]), Y[i] = f_even(A[i])
        vector<ull> X(L), Y(L);
        // 当牌堆数为1时, 唯一牌必然为1
        X[L-1] = 1;
        Y[L-1] = 1;
        for(int i = L - 2; i >= 0; i--){
            // 奇数轮变换: X[i] = 2 * Y[i+1] - 1
            X[i] = 2 * Y[i+1] - 1;
            // 偶数轮变换, 根据 A[i] 的奇偶性
            if(A[i] % 2 == 0){
                // A[i] 为偶, 此时 A[i+1] = A[i] / 2
                Y[i] = 2 * X[i+1];
            } else {
                // A[i] 为奇, 此时 A[i+1] = (A[i] + 1) / 2
                ull m = A[i+1];
                if(X[i+1] < m)
                    Y[i] = 2 * X[i+1];
                else
                    Y[i] = 2 * m - 1;
            }
        }
        cout << X[0] << "\n";
    }
    return 0;
}
```

Summary

- 将洗牌过程看作交替进行的“压缩”操作, 每轮操作均将牌堆数变为 $\text{ceil}(n/2)$;
- 分别分析奇数轮 (取左侧牌) 和偶数轮 (取右侧牌) 的变换关系, 建立递归关系;
- 构造自底向上的链, $O(\log n)$ 时间内求出最终 $f_{\text{odd}}(n)$ 即为答案。

F、翻牌游戏

考虑倒推。如果现在正面朝上的数不含 x ，答案就为0，否则就翻转这张正面为 x 的牌。设翻完之后新的朝上的数为 y 。如果其他牌向上的数没有 y ，就得到答案为1了，否则就找到冲突的那张牌，将其翻转。重复这一过程，直到没有冲突为止。

每次询问倒推一下的话，复杂度最坏为 $O(nq)$ ——考虑 n 张牌，第 i 张牌正面是 i ，反面是 $i + 1$ ，询问时每个点都问一遍。

所以考虑优化。可以在进行上述过程的同时进行记忆化处理。

出题人的做法是，让没有出现过的数进入队列，进行广度优先搜索，这样可以在线性复杂度内预处理出每个 x 的答案，查询时 $O(1)$ 回答即可。这样避免在处理一些有环的数据时出错。

复杂度错误或者dfs写的不对，导致一直在一个环里转圈，都可能导致RE或TLE。

```
#include<bits/stdc++.h>
#define fi first
#define se second
using namespace std;
typedef pair<int,int> pii;
const int maxn=5e5+10;
int n,q,a[maxn],b[maxn],exi[maxn],vis[maxn],ans[maxn];
vector<int> ver[maxn];
int main(){
    ios::sync_with_stdio(0);
    cin>>n;
    for(int i=1;i<=n;i++){
        cin>>a[i];
        ans[a[i]]=-1;
        exi[a[i]]=1;
    }
    queue<pii> Q;
    for(int i=1;i<=n;i++){
        cin>>b[i];
        if(!exi[b[i]]){
            ans[b[i]]=0;
            Q.push({b[i],0});
        }
        ver[a[i]].push_back(b[i]);
        ver[b[i]].push_back(a[i]);
    }
    while(Q.size()){
        auto [num,val]=Q.front(); Q.pop();
        if(ans[num]==-1){
            ans[num]=val;
        }
        if(vis[num]) continue;
        vis[num]=1;
        for(auto u:ver[num]){
            Q.push({u,val+1});
        }
    }
    cin>>q;
    for(int i=1;i<=q;i++){
        int x; cin>>x;
        cout<<ans[x]<<" \n"[i==q];
    }
    return 0;
}
```

G、大数模7

不能直接算。把输入的数用字符串存下来，一位一位处理。可以发现 $10^k \bmod 7$ 是有周期的，可以每次乘以10，再加上那一位的数字，也可以一位一位用快速幂暴力算。

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    string ch;
    cin>>ch;
    int n=ch.length();
    int ans=0;
    for(int i=0;i<n;i++){
        ans*=3;
        ans+=ch[i]-'0';
        ans%=7;
    }
    cout<<ans<<"\n";
    return 0;
}
```

H-小树的分数

每次分数变化前先sort一下，别让最高分的人变化，变化另外两个人中的一个。注意一下不能连续的性质，if一下。

```

#include <bits/stdc++.h>

using namespace std;

void solve(){
    int n;
    vector<pair<int,char>> S(3);
    S[0].second = 'A';
    S[1].second = 'B';
    S[2].second = 'C';
    cin >> n >> S[0].first >> S[1].first >> S[2].first;
    vector<int> v(n);
    for (int i = 0; i < n; i++){
        cin >> v[i];
    }
    vector<char> ans;
    for (int i = 0; i < n; i++){
        sort(S.begin(),S.end());
        if (v[i] > 0){
            if (ans.size() != 0 && ans.back() == S[2].second) {
                ans.push_back(S[1].second);
                S[1].first += v[i];
            } else {
                ans.push_back(S[2].second);
                S[2].first += v[i];
            }
        } else {
            if (ans.size() != 0 && ans.back() == S[0].second) {
                ans.push_back(S[1].second);
                S[1].first += v[i];
            } else {
                ans.push_back(S[0].second);
                S[0].first += v[i];
            }
        }
    }
    for (auto x : ans){
        cout << x;
    }
    cout << "\n";
}

int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    int T;cin >> T;
    while(T--){
        solve();
    }
    return 0;
}

```

I-小树的积分

积分式子积出来是 $\frac{\pi^2}{4}$

或者按照微积分的定义做，每次取一小段 (0.01其实就够了)，把长条矩形的面积加起来，精度只要求到小数点后三位，基本上方法对了不会错。